

Rapport Brief 3

Mise en place d'une solution de gestion de données et d'une interface analytique polyvalente

Formation : Data Engineer P1 – 2025–2027



Réalisé par :

- Caroline berge
- Kaouter RHAZLANI
- Zoubir MABED

Le 15 Janvier 2026

Plan

Plan.....	1
1. Introduction.....	2
1.1. Contexte du projet.....	2
1.2. Objectifs du projet.....	2
2. Analyse des données sources.....	3
3. Modèle logique de données.....	4
4. Intégrité des données et cohérence métier :.....	7
5. Architecture ETL.....	8
5.1. Extraction (Bronze).....	8
5.2. Transform – Validation en deux passes & nettoyage.....	9
Validation des données avec Pandera.....	9
Stratégie de validation en deux passes.....	9
5.3. Chargement (Load).....	11
6. Technologies utilisées :.....	12
7. Structure du projet.....	14
8. Traçabilité et journalisation des actions.....	14
La table audit_log assure une traçabilité complète de toutes les interactions avec la base de données sauvetage_db. Elle enregistre pour chaque opération :.....	14
Cette journalisation systématique permet de garantir :.....	15
Ainsi, audit_log constitue un outil central pour la sécurité, la fiabilité et la transparence de la base de données.....	15
9. Application Streamlit : Interface CRUD & Analytics.....	16
Pages principales.....	16
KPI & requêtes analytiques :.....	22
10. Tests unitaires :.....	24
11. Conclusion & perspectives.....	25
Perspectives et Évolutions Futures.....	25



1. Introduction

1.1. Contexte du projet

Le département des Opérations Maritimes (SECMAR) a rencontré des problèmes techniques ayant entraîné la perte partielle ou l'incohérence de ses données historiques de sauvetage maritime.

Les seules sources exploitables récupérées sont des fichiers CSV hétérogènes, non normalisés et présentant de nombreuses anomalies (types incorrects, valeurs négatives, doublons, champs incohérents).

Ces données n'étaient pas exploitables directement ni pour l'analyse, ni pour un usage métier.

1.2. Objectifs du projet

L'objectif principal est de reconstruire une base de données fiable, cohérente et exploitable, tout en fournissant une interface simple pour les équipes métier.

l'objectif du projet consiste à :

- Centraliser les données dans PostgreSQL comme source unique de vérité
- Normaliser les formats, types et règles métier
- Mettre en place une validation en deux passes :
 - validation souple (lazy) pour identifier les erreurs,
 - validation stricte (fast-fail) avant chargement
- Assurer une traçabilité complète via une table audit
- Développer une interface Streamlit :
 - CRUD métier,
 - dashboards analytiques,
 - visualisation du schéma et audit
- Garantir la qualité via des tests unitaires.

2. Analyse des données sources

Le projet repose sur l'exploitation de quatre fichiers CSV principaux, liées via un identifiant commun **operation_id**, permettant de relier l'ensemble des informations relatives aux opérations de sauvetage maritime. Chaque fichier joue un rôle spécifique dans le modèle de données et présente des enjeux de qualité nécessitant un traitement ETL dédié :

- **operations.csv** (~15 000 lignes, 26 colonnes) : contient les informations générales de chaque opération (dates, CROSS, type d'alerte, zone). Les principaux problèmes identifiés sont des operation_id invalides ou négatives, des formats de dates hétérogènes et des champs textuels bruités.
- **operations_stats.csv** (~15 000 lignes, plus de 60 colonnes) : regroupe les statistiques détaillées par opération (nombre de personnes impliquées, distances parcourues, coefficients de marée, coordonnées géographiques). Ce fichier présente des incohérences de plages de valeurs, des coordonnées géographiques bruitées ainsi que des dates éclatées sur plusieurs champs.
- **flotteurs.csv** (~8 500 lignes, 7 colonnes) : décrit les flotteurs et moyens nautiques engagés dans les opérations (type, catégorie, pavillon, résultat). Les anomalies relevées incluent des doublons, des incohérences dans les numéros d'ordre et des valeurs hors du domaine métier.
- **resultats_humain.csv** (~18 000 lignes, 4 colonnes) : fournit le bilan humain par opération (personnes secourues, blessées, décédées, catégorie de personnes). Les données comportent des valeurs négatives et des catégories non standardisées.

L'ensemble de ces constats a conduit à la mise en place d'un processus de nettoyage, de normalisation et de validation des données afin d'assurer la cohérence inter-table et la fiabilité des analyses produites.

3. Modèle logique de données

La base de données repose sur un modèle relationnel structuré autour de tables principales et de tables de support.

Tables principales :

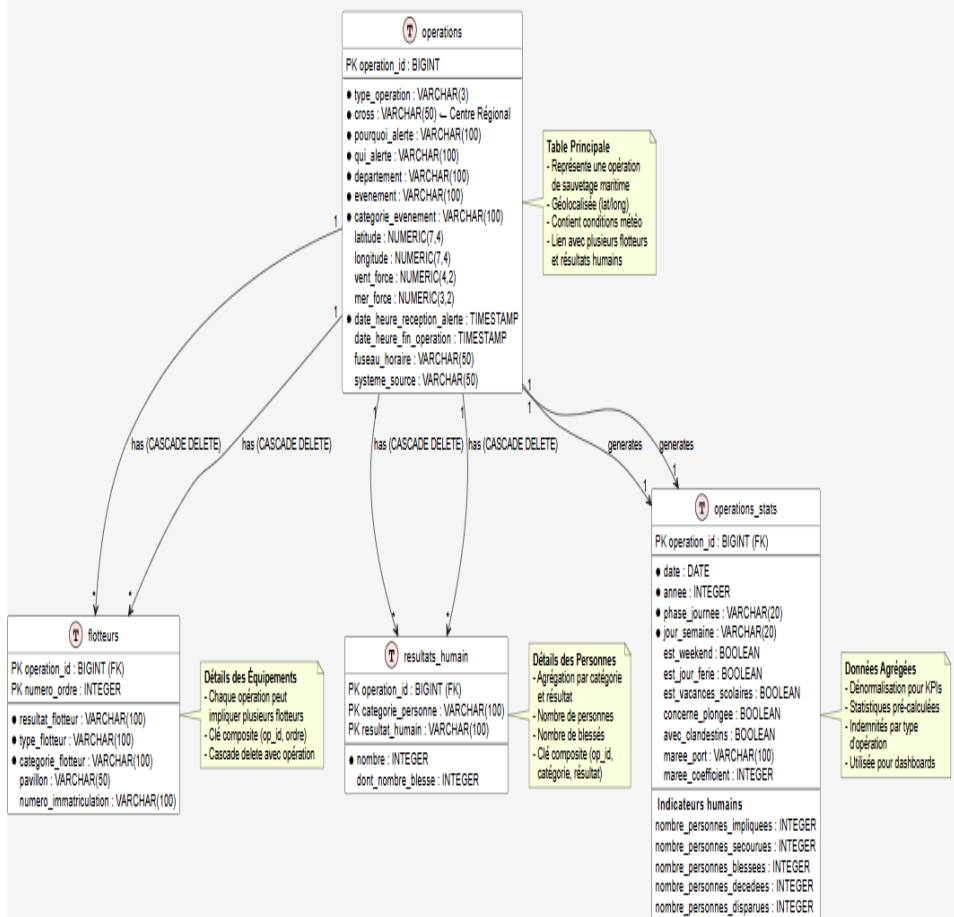
La table operations constitue l'entité centrale du modèle et est identifiée par une clé primaire operation_id. Elle est référencée par les autres tables métier afin d'assurer la cohérence globale des données. Les tables principales sont :

- operations (PK : operation_id)
- flotteurs (PK composite : operation_id + numero_ordre)
- resultats_humain (FK → operations)
- operations_stats (FK → operations)

Tables de support :

Des tables de support complètent le modèle afin d'améliorer la normalisation et la maintenabilité du système. La table reference_lists centralise les référentiels métiers (CROSS, types d'événements, autorités, catégories de flotteurs), tandis que la table audit_log permet une traçabilité complète des opérations sur les données, renforçant la transparence, le suivi des modifications et la conformité aux exigences de gouvernance des données. Les tables de supports sont :

- reference_lists (CROSS, événements, autorités, flotteurs)
- audit_log (traçabilité complète)



reference_lists
● id : SERIAL [PK] ● category : VARCHAR(100) [NOT NULL] ● value : VARCHAR(200) [NOT NULL] ● display_order : INTEGER DEFAULT 0 ● is_active : BOOLEAN DEFAULT TRUE ● description : TEXT ● created_at : TIMESTAMP ● updated_at : TIMESTAMP
[CONSTRAINTS: UNIQUE(category, value)] [TRIGGER: update_updated_at_column] [INDEXES: category, is_active]
CATEGORIES (29 total) • type_operation (5 vals) • pavillon (2 vals) • categorie_flotteur (6 vals) • resultat_flotteur (9 vals) • categorie_personne (10 vals) • resultat_humain (9 vals) • vent_direction_categorie (8 vals) • systeme_source (2 vals: secmarweb, seamis_json) • prefecture_maritime (3 vals: manche, atlantique, mediterranee) • maree_categorie (4 vals: 20-45, 46-70, 71-95, 96-120) • cross (7 vals: Gris-Nez, Jobourg, Cersen, Etel, Med, Antilles-Guyane, Réunion)

audit_log
● id : SERIAL [PK] ● table_name : TEXT [NOT NULL] ● action : TEXT [NOT NULL] ● record_id : TEXT [NOT NULL] ● user_name : TEXT DEFAULT 'system' ● old_values : JSONB ● new_values : JSONB ● details : TEXT ● sql_query : TEXT ● created_at : TIMESTAMP DEFAULT NOW
[INDEXES: table_name, action, created_at_DESC, record_id] [ACTIONS TRACKED: INSERT, UPDATE, DELETE, VIEW] ""Traçabilité toutes tables principales""

TRAÇABILITÉ IMMuable

Compliance légale
 Historique complet INSERT/UPDATE/DELETE
 Enregistrement user_name
 JSON pour old/new values
 4+ Indexes pour requêtes rapides



4. Intégrité des données et cohérence métier :

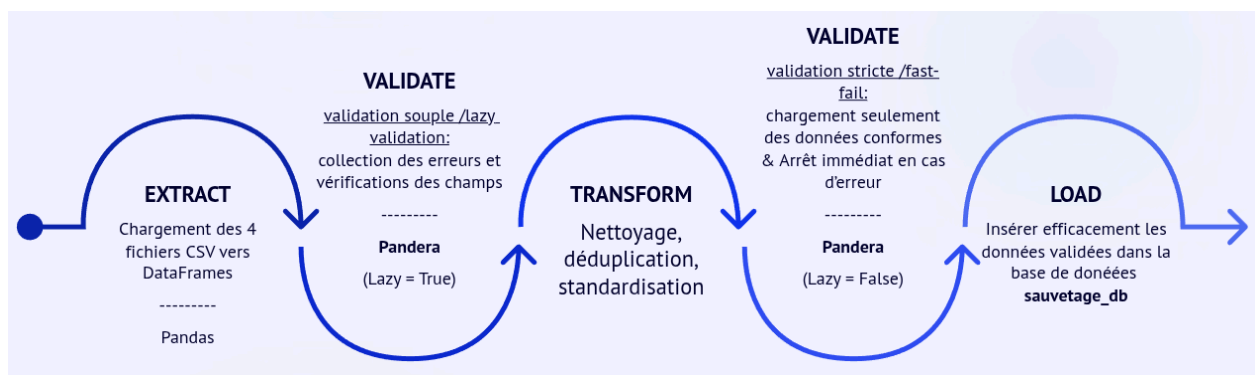
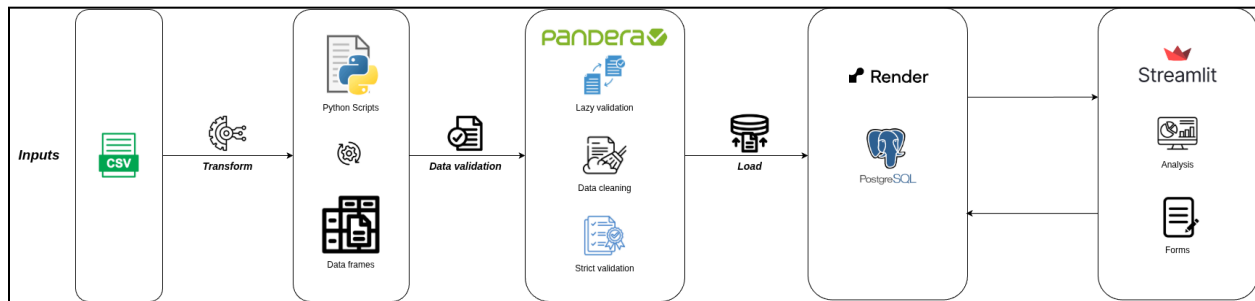
L'intégrité des données est assurée par la mise en place de règles strictes d'intégrité référentielle, de contraintes d'unicité et de contrôles de cohérence métier.

Ainsi, aucun flotteur ne peut exister sans être rattaché à une opération, garantissant que chaque équipement est associé à un contexte opérationnel valide. De la même manière, aucun bilan humain ne peut être enregistré sans référence explicite à une opération existante, évitant toute donnée orpheline.

Des contraintes d'unicité sont également appliquées afin d'éviter les doublons : un flotteur est identifié de manière unique par le couple (operation_id, numero_ordre), tandis qu'un bilan humain est unique pour une combinaison donnée de catégorie et de résultat au sein d'une opération.

Enfin, la cohérence métier est renforcée par des règles de validation temporelle et logique. La date de fin d'une opération ne peut être antérieure à sa date de début ($\text{date_fin} \geq \text{date_debut}$), et les totaux statistiques sont systématiquement vérifiés afin d'être cohérents avec les résultats détaillés du bilan humain. Ces contrôles garantissent la fiabilité, la consistance et l'exploitabilité des données tout au long du pipeline de traitement.

5. Architecture ETL



Le pipeline suit les étapes classiques **Extract → Transform → Load**, inspirée du modèle **Bronze / Silver / Gold**.

5.1. Extraction (Bronze)

La phase d'ingestion repose sur la centralisation des différentes sources de données, avec le chargement des quatre fichiers CSV bruts depuis un répertoire unique. La lecture des données est réalisée de manière homogène grâce à l'utilisation d'une fonction générique permettant de standardiser l'import de fichiers issus de différentes sources (CSV, API, Parquet).

La portabilité du code est assurée par une gestion dynamique des chemins d'accès, garantissant une exécution fiable sur plusieurs environnements (local, collaboratif ou déployé). Les jeux de données sont ensuite structurés et regroupés au sein d'un dictionnaire de DataFrames pandas, facilitant leur manipulation et leur transmission aux étapes suivantes du pipeline.



À ce stade, aucune transformation ni correction n'est appliquée : les données sont conservées dans leur état brut afin de préparer de manière rigoureuse les phases ultérieures de nettoyage, de validation et de transformation du pipeline ETL.

5.2. Transform – Validation en deux passes & nettoyage

Validation des données avec Pandera

La couche Silver est dédiée au nettoyage et à la validation des données afin de garantir leur qualité, leur cohérence et leur conformité aux règles métier. Cette étape repose sur l'utilisation de Pandera, une bibliothèque Python spécialisée dans la validation de données tabulaires manipulées sous forme de DataFrames pandas.

Pandera permet de définir des contrats de données (data contracts) via des schémas déclaratifs, décrivant précisément la structure attendue, les types, les contraintes et les règles métier associées aux données.

Schémas Pandera définis

Les règles de validation sont formalisées à travers des schémas Pandera spécifiques à chaque table métier :

- schema_operations
- schema_operations_stats
- schema_flotteurs
- schema_resultats_humain

Ces schémas constituent le référentiel de qualité des données et assurent une validation cohérente et reproductible tout au long du pipeline ETL.

Stratégie de validation en deux passes

Afin de concilier robustesse du pipeline et transparence sur la qualité des données sources, une stratégie de validation en deux niveaux a été mise en place.

- **Validation souple (Lazy validation)**

Une validation strictement bloquante dès l'ingestion aurait entraîné l'interruption du pipeline tout en masquant l'ampleur réelle des problèmes de qualité des données. Pour éviter cela, une première validation souple est appliquée à l'aide de Pandera avec le paramètre `lazy=True`.



Cette approche permet de :

- identifier l'ensemble des erreurs présentes dans les données sans bloquer le pipeline ;
 - collecter toutes les violations de règles de validation en une seule exécution
 - extraire les lignes non conformes dans un répertoire dédié (data/rejected) ;
 - autoriser la poursuite du pipeline pour les données exploitables ;
 - associer chaque erreur à la règle métier ou technique violée, assurant une transparence totale sur la qualité des données sources.
- **Nettoyage des données**

À l'issue de la validation souple, des opérations de nettoyage sont appliquées sur les données exploitables afin d'améliorer leur qualité et leur homogénéité. Les principales transformations incluent :

- la normalisation des chaînes de caractères ;
- la suppression des doublons ;
- la correction des formats (dates, types numériques, etc.) ;
- le filtrage des valeurs hors domaine ;
- l'exclusion des operation_id négatifs.

Chaque jeu de données dispose de sa fonction de nettoyage dédiée, garantissant une logique claire et maintenable :

- clean_operations
 - clean_operations_stats
 - clean_flotteurs
 - clean_resultats_humain
- **Validation stricte (Fast-fail)**

Une fois les données nettoyées, une seconde validation stricte est appliquée à l'aide de Pandera avec lazy=False. Dans ce mode fast-fail, la moindre non-conformité entraîne l'échec immédiat de la validation.

Cette étape garantit que :

- Le schéma est respecté de manière stricte ;
- Seules les données parfaitement conformes sont conservées ;
- Les données sont prêtes pour l'insertion en base PostgreSQL, sans risque d'erreur ou d'incohérence.



5.3. Chargement (Load)

Insertion des données en base PostgreSQL

Les données nettoyées et validées sont insérées dans la base de données PostgreSQL via un chargement massif utilisant la commande COPY, permettant une ingestion rapide et efficace de volumes importants de données. Les opérations d'insertion sont encapsulées dans des transactions atomiques, garantissant la cohérence de la base : en cas d'erreur, l'ensemble de l'opération est annulé afin d'éviter tout état intermédiaire incohérent.

La modélisation repose sur une structure relationnelle simple, s'appuyant sur des clés primaires et des clés étrangères définies autour de `operation_id`, assurant l'intégrité référentielle entre les différentes tables métier. Enfin, une indexation ciblée des colonnes critiques est mise en place afin d'optimiser les performances des requêtes analytiques et opérationnelles, notamment lors des jointures et des filtres fréquents.

6. Technologies utilisées :

PostgreSQL : est un système de gestion de base de données relationnelle (SGBDR) open-source, reconnu pour sa robustesse, sa fiabilité et ses performances dans le traitement de données tabulaires. Dans le cadre de ce projet ETL footballistique, PostgreSQL a été choisi pour les raisons suivantes :

- **Données fortement structurées** : idéale pour organiser les informations des matchs, équipes, stades et compétitions.
- **Intégrité référentielle** : garantit la cohérence entre les tables liées (par exemple, les relations entre équipes et matchs).
- **Requêtes analytiques performantes** : permet d'effectuer des agrégations et des analyses statistiques rapidement.
- **Intégration avec Python** : compatible avec les bibliothèques Python telles que psycopg2, SQLAlchemy et pandas, facilitant le développement ETL et l'exploitation des données.

Hébergement cloud via Render : offre une solution clé-en-main pour déployer la base de données sans gestion complexe de l'infrastructure.

Les avantages d'utilisation du render :

- Instance PostgreSQL managée
- Accès réseau sécurisé
- Backups et snapshots
- Stockage persistant

Streamlit : Streamlit est un framework Python open source permettant de créer rapidement des applications web interactives orientées données, sans nécessiter de compétences en développement front-end (HTML, CSS, JavaScript).

Dans ce projet, Streamlit a été utilisé pour :

- Créer une interface analytique permettant de visualiser les données issues des opérations de surveillance et de sauvetage
- Mettre à disposition un outil CRUD (Create, Read, Update, Delete) pour interagir directement avec la base de données PostgreSQL
- Permettre à des utilisateurs non techniques de consulter, modifier et enrichir les données
- Afficher des tableaux interactifs, des indicateurs clés (KPI) et des pages thématiques

- Visualiser le schéma de la base de données et l'historique des opérations via une page dé

Pandera : Pandera est une bibliothèque Python dédiée à la validation de données tabulaires (DataFrames pandas). Elle permet de définir un contrat de données (data contract) sous forme de schémas déclaratifs. L'objectif est de garantir la qualité, la cohérence et la conformité métier des données.

Git / GitHub : Dans ce projet, une stratégie Git simple et professionnelle a été mise en place afin de faciliter le travail en équipe et de limiter les conflits.

Organisation des branches:

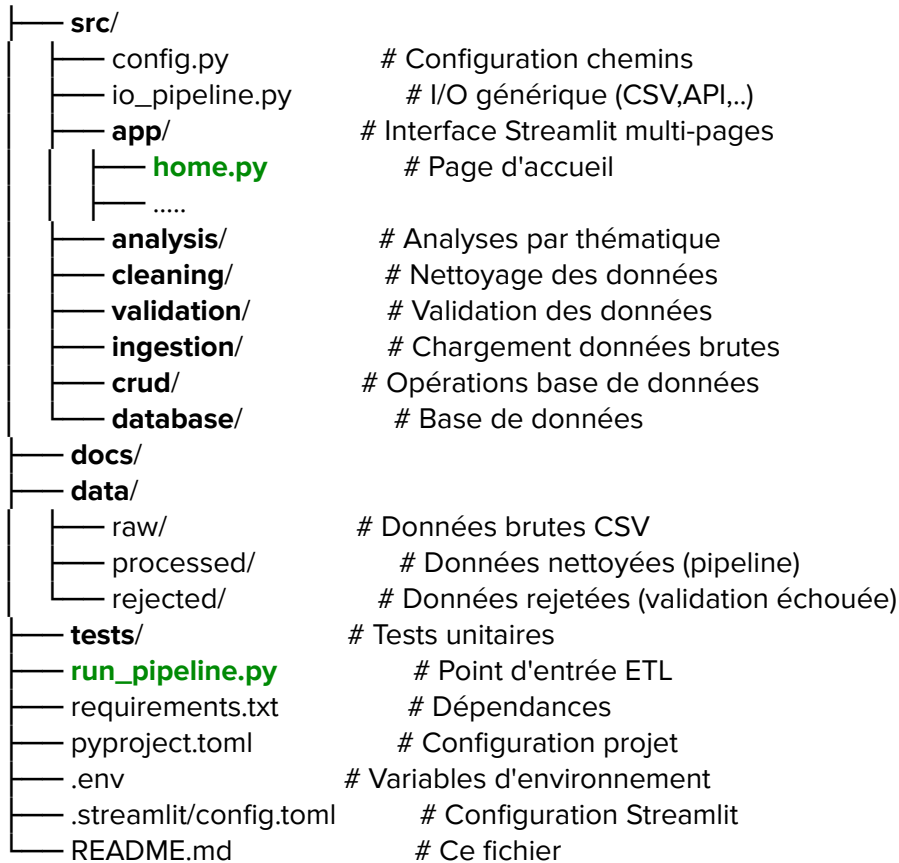
- **main** : Branche stable contenant la version finale et fonctionnelle du projet
- **dev** : Branche d'intégration utilisée pour regrouper et tester les fonctionnalités avant validation finale
- **Branches de fonctionnalités (feature/*)** : Chaque fonctionnalité est développée dans une branche dédiée, par exemple :
 - feature/ingestion
 - feature/validation
 - feature/database
 - feature/crud

Workflow suivi :

- Création d'une branche feature à partir de dev
- Développement de la fonctionnalité (ingestion, validation, CRUD, etc.)
- Commits clairs et explicites (ex. feat: add csv ingestion)
- Création d'une Pull Request de la branche feature vers dev
- Revue du code par au moins un membre de l'équipe
- Fusion dans dev après validation
- Fusion finale de dev vers main pour livrer une version stable

7. Structure du projet

Brief-3-Gestion-donnees-et-UI-analytique-polyvalente-Grass-Squad/



8. Traçabilité et journalisation des actions

La table `audit_log` assure une traçabilité complète de toutes les interactions avec la base de données `sauvetage_db`. Elle enregistre pour chaque opération :

- l'identifiant de l'opération concernée,
- la date et l'heure de l'action,
- le type d'action réalisée (INSERT, UPDATE, DELETE),
- la table ciblée par l'action,
- l'identifiant de l'enregistrement modifié,
- les valeurs avant et après la modification,
- la requête SQL exécutée, sous forme de chaîne de caractères, l'utilisateur ayant effectué l'opération.



Cette journalisation systématique permet de garantir :

- la conformité aux exigences de gouvernance et de contrôle des données,
- le debugging en cas de problèmes ou de comportements inattendus,
- la relecture historique des modifications pour analyser l'évolution des données dans le temps.

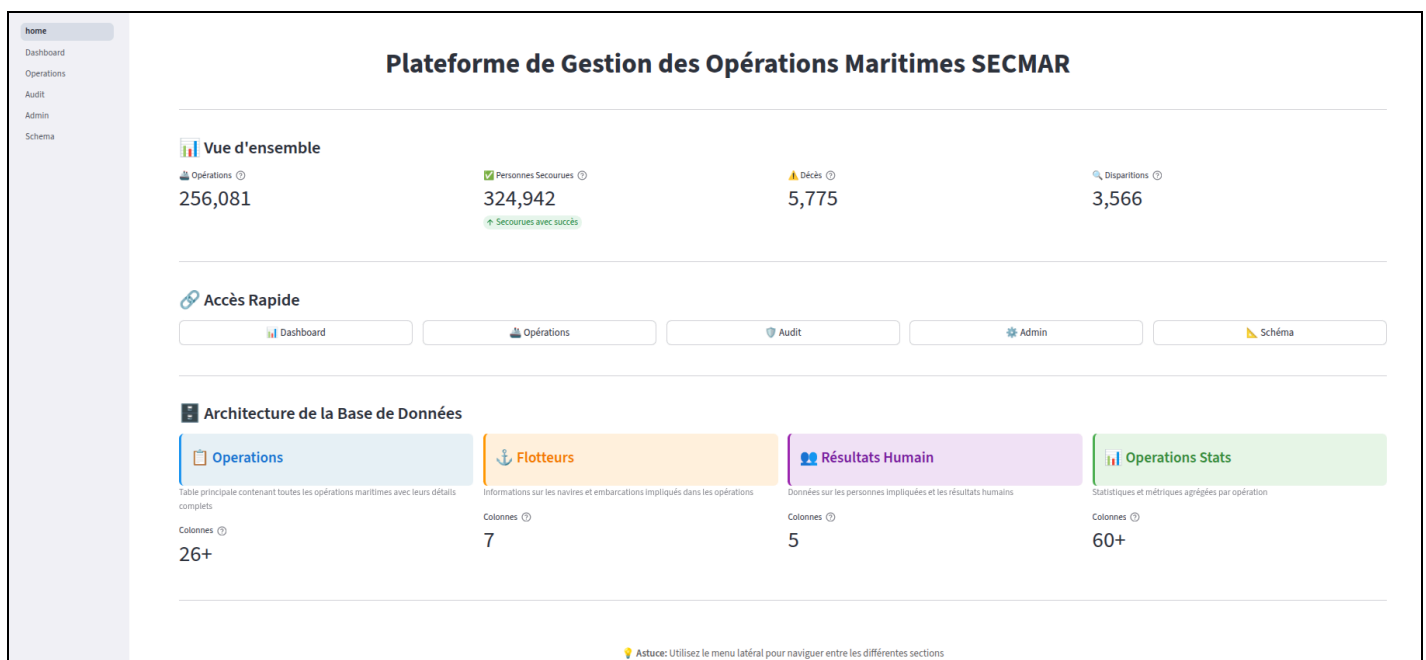
Ainsi, `audit_log` constitue un outil central pour la sécurité, la fiabilité et la transparence de la base de données.

9. Application Streamlit : Interface CRUD & Analytics

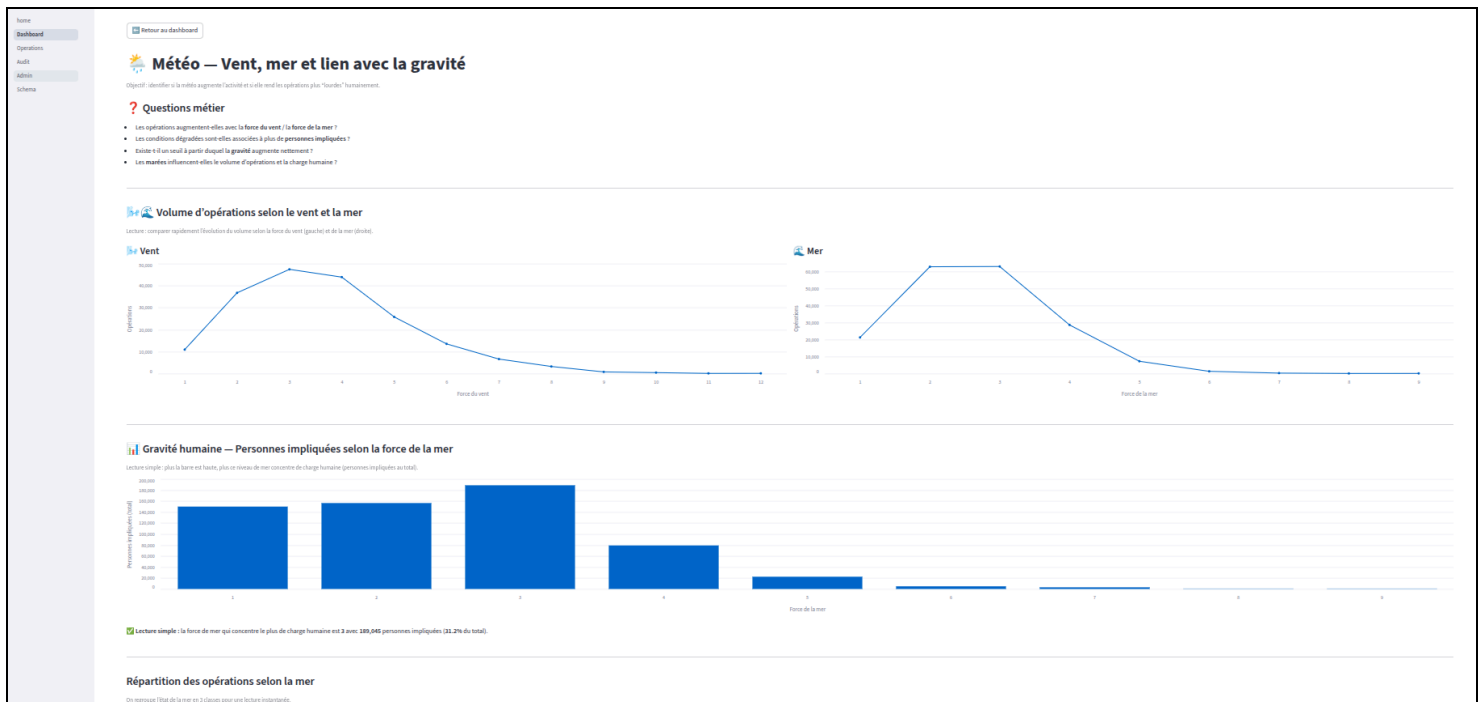
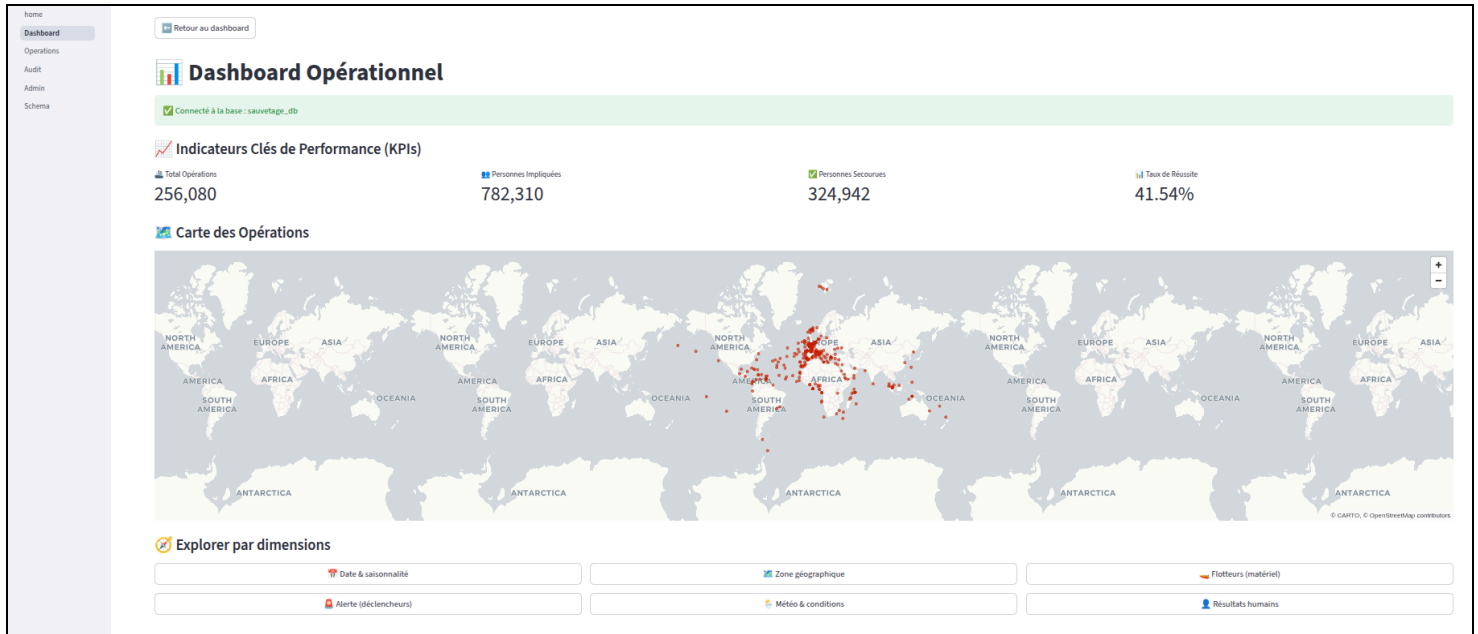
L'application SECMAR, développée avec Streamlit, offre une interface intuitive permettant aux utilisateurs non techniques d'exploiter les données maritimes de manière interactive, via CRUD, tableaux de bord et analyses.

Pages principales

- **Home** : KPI globaux et navigation vers les sections principales.



- **Dashboard** : Analyses par axes : alertes, zones géographiques, temporalité, flotteurs, résultats humains.



- **Operations** : Gestion complète des opérations (CRUD) avec filtres multi-critères, tri et pagination.

Gestion des Opérations

Liste des Opérations

Rechercher par opération_id

Tableau des Opérations

ID	CROSS	Estimation	Date/Heure	Dépt	Coordonnées	Actions
110420241728	Est		2026-01-16 13:03			[Edit] [Delete] [Details]
110420241729	Est	Transport sanitaire	2026-01-16 13:03	Beckham	47.84, 1.40	[Edit] [Delete] [Details]
110420241730	Conen	Explosion de rouillage	2026-01-16 13:03	Kissida	5.05, 5.09	[Edit] [Delete] [Details]
110420241731	Polyside	Situation indétectée...	2026-01-16 13:03	Polyside	12.05, 146.24	[Edit] [Delete] [Details]
110420241732	Conen	Incendie d'urgence à la...	2026-01-16 13:03	Kissida	48.25, 4.25	[Edit] [Delete] [Details]
110420241733	Conen	Situation indétectée...	2026-01-16 13:03	Frederic	48.25, 4.12	[Edit] [Delete] [Details]
110420241734	Polyside	Situation indétectée...	2026-01-16 13:03	Polyside	12.05, 146.24	[Edit] [Delete] [Details]
110420241735	Est	Avion d'urgence	2026-01-16 13:03	Londre	48.25, 1.38	[Edit] [Delete] [Details]
110420241736	Est	Avion d'urgence	2026-01-16 13:03	Grenoble	48.25, 1.07	[Edit] [Delete] [Details]
110420241737	Est	Incendie	2026-01-16 13:03	Grenoble	48.25, 1.38	[Edit] [Delete] [Details]

256081 10

Gestion des Opérations

Modifier l'Opération #110420241728

Revenir à la liste

Modifier les données de l'opération

Rechercher par opération_id

opération_id	type_opération	responsable	date_heure	dept	coordonnées	status	commentaire
110420241728	Est	Est	2026-01-16 13:03	Beckham	47.84, 1.40	En cours	

Gestion des Opérations

Consultation de l'Opération #110420241728

Revenir à la liste

Rechercher par opération_id

Estimation: N/A

Date: 2026-01-16 13:03

Données complètes de l'opération

opération_id	type_opération	responsable	date_heure	dept	coordonnées	status	commentaire
110420241728	Est	Est	2026-01-16 13:03	Beckham	47.84, 1.40	En cours	

Historique d'audit de cette opération

Date	Heure	Opération	Utilisateur	Action	Commentaire
2026-01-16 13:03:03	13:03	Consultation de l'opération 110420241728	Admin	Consultation de l'opération 110420241728	
2026-01-16 13:03:04	13:03	Modification de l'opération 110420241728	Admin	Modification de l'opération 110420241728	
2026-01-16 13:03:05	13:03	Modification de l'opération 110420241728	Admin	Modification de l'opération 110420241728	
2026-01-16 13:03:06	13:03	Modification de l'opération 110420241728	Admin	Modification de l'opération 110420241728	
2026-01-16 13:03:07	13:03	Modification de l'opération 110420241728	Admin	Modification de l'opération 110420241728	
2026-01-16 13:03:08	13:03	Modification de l'opération 110420241728	Admin	Modification de l'opération 110420241728	
2026-01-16 13:03:09	13:03	Modification de l'opération 110420241728	Admin	Modification de l'opération 110420241728	
2026-01-16 13:03:10	13:03	Modification de l'opération 110420241728	Admin	Modification de l'opération 110420241728	

- **Audit** : Historique des modifications (avant/après), export CSV pour relecture et traçabilité.

[illegible]

- Admin : Gestion centralisée des listes de référence.

Home

Dashboard

Operations

Alerts

Admin

Schema

Administration - Listes de Référence

Listes de Référence

Gestion des Listes de Référence

Initialisation des listes de référence

Cette action va exécuter le script SQL pour créer/initialement la table reference_liste

Cette opération va :

- Supprimer la table reference_liste, si elle existe
- Créer une nouvelle table reference_liste
- Insérer toutes les valeurs par défaut
- Attention : Les valeurs personnalisées seront perdues !

Initialiser les listes de référence

Sélectionnez une catégorie pour gérer ses valeurs

catégorie_flotteur

Catégories de flotteurs (Compass, PMS, etc.)

8 valeurs()

Gérer

catégorie_personne

Catégories de personnes (Ingénieur, etc.)

10 valeurs()

Gérer

CROSS

Centres d'analyse opérationnels de surveillance et de sauvetage

7 valeurs()

Gérer

marée_catégorie

Catégories de coefficients de marée

4 valeurs()

Gérer

pavillon

Pavillons des navires (Français, Étranger)

2 valeurs()

Gérer

prefecture_maritime

Préfectures maritimes

3 valeurs()

Gérer

resultat_flotteur

Résultats possibles pour les flotteurs

3 valeurs()

Gérer

resultat_humain

Résultats humains (Secours, Décédé, etc.)

4 valeurs()

Gérer

systeme_source

Systèmes sources des données

2 valeurs()

Gérer

type_operation

Type d'opération (SAR, MMSI, etc.)

5 valeurs()

Gérer

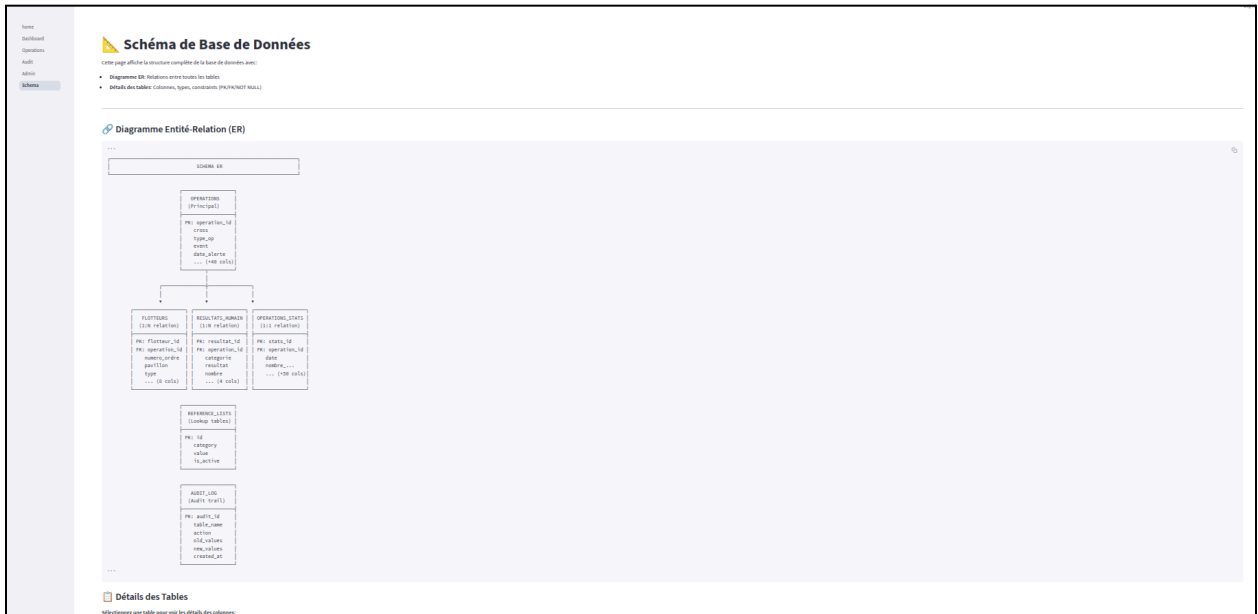
vent_direction_catégorie

Directions cardinales du vent

4 valeurs()

Gérer

20



Détails des Tables

Sélectionnez une table pour voir les détails des colonnes:

Choisir une table

OPERATIONS - Table principale - Opérations de sauvetage maritimes

OPERATIONS

Table principale - Opérations de sauvetage maritimes

Colonne	Type	PK	NB	Statut	Description
operation_id	INTEGER				Identifiant unique de l'opération
cnic	VARCHAR(255)				Certificat d'identité opérationnel
type_operation	VARCHAR(255)			✓	Type d'opération (SAR, MMSI, SAR, PLS, etc.)
latitude	VARCHAR(255)			✓	Type d'équipement
categorie_sauvetage	VARCHAR(255)			✓	Catégorie d'équipement
date_heure_creation_alerte	TIMESTAMP			✓	Date et heure de l'alerte
date_heure_fin_operation	TIMESTAMP			✓	Date et heure de fin
priorite_alerte	TEXT			✓	Priorité de l'alerte
message_alerte	VARCHAR(255)			✓	Message d'alarme (SMS, téléphone, etc.)
qst_alerte	VARCHAR(255)			✓	Qui a envoyé l'alerte

Colonnes

28

1

0

Colonne(s) manquante(s)

Colonne(s) manquante(s)

Informations en Direct de la Base de Données

Nombre de lignes par table:

Table	Lignes
operations	20000
Bureau	10000
historique_alertes	20000
operations_cnic	10000
statistiques	7
reference_data	10

Taille des tables:

Table	Taille
operations	120 MB
Bureau	60 MB
historique_alertes	120 MB
operations_cnic	120 MB
statistiques	30 KB
reference_data	10 KB

Légende

Opérations:

- ✓ Colonne PK
- ✓ Colonne FK
- ✓ Accepter les valeurs NULL

Types de données existants:

- INTEGER: Entiers entiers
- VARCHAR(n): Texte de longueur variable
- NUMERIC(p,s): Nombre décimal
- DECIMAL(p,s): Nombre décimal

Types de données (JDBC):

- NUMBER: Valeur numérique
- DATE: Date uniquement
- TEXT: Texte long
- JDBC: Données JDBC

Règles techniques:

- Les opérations 1 à 10 sont jugées prioritaires via les alertes (JDBC)
- La relation 1 à 1 entre operations et operations_cnic est unidirectionnelle.
- Le champ date_heure_creation_alerte ne doit pas être nul.
- Le champ message_alerte ne doit pas être nul.
- Le champ qst_alerte ne doit pas être nul.



Architecture

L'application suit le modèle MVC (Model-View-Controller) :

- **Model (Données)** : Fichiers CRUD et accès PostgreSQL (operations_crud.py, flotteurs_crud.py, audit_crud.py).
- **View (Présentation)** : Pages, vues métier et composants réutilisables pour afficher données et formulaires.
- **Controller (Logique métier)** : Coordination entre View et Model, gestion des utils, formatting et st.session_state.

Fonctionnement et points clés :

- Les interactions utilisateur déclenchent la réexécution complète du script Streamlit, tout en conservant l'état via st.session_state.
- Réutilisabilité et modularité : Pages, vues et composants séparés pour faciliter maintenance et évolution.
- Performance : Cache centralisé, pagination, et requêtes optimisées pour un affichage rapide.
- Sécurité et audit : Toutes les actions critiques sont journalisées dans audit_log pour traçabilité, conformité et debugging.
- Extensibilité : Nouvelles pages et vues métier facilement ajoutables sans refactorisation du code existant.

En résumé, SECMAR combine simplicité d'utilisation, fiabilité des données et flexibilité technique, permettant aux utilisateurs métiers de consulter, analyser et gérer efficacement les opérations maritimes.

KPI & requêtes analytiques :

L'application SECMAR fournit des tableaux de bord avancés pour l'analyse multidimensionnelle des opérations maritimes. Les principaux KPI incluent :

- Nombre total d'opérations uniques,
- Personnes secourues, décédées ou disparues,
- Flotteurs impliqués par catégorie (commerce, pêche, plaisance),
- Taux d'intervention par zone maritime,
- Top événements et catégories d'événements,
- Moyens d'alerte utilisés.

Les requêtes analytiques permettent de croiser les données selon différentes dimensions :

- 
- Temporelles : année, mois, jour, phase de journée, saisonnalité,
 - Géographiques : CROSS, département, zones STM/DST,
 - Métiers : type d'opération, catégorie d'alerte, pavillon.

La table `operations_stats` centralise ces indicateurs avec des métriques détaillées, incluant notamment les résultats humains avec ou sans clandestins, les flotteurs selon leur type, et les conditions météorologiques. Ces analyses alimentent six vues spécialisées : alertes, saisonnalité, zones géographiques, flotteurs, météo et résultats humains, offrant ainsi une vision 360° des opérations de sauvetage maritime.

Pour garantir des performances optimales, les requêtes analytiques sont mises en cache, permettant un rendu rapide et interactif des dashboards.

10. Tests unitaires :

Pour garantir la qualité, l'intégrité et la fiabilité des données, plusieurs tests unitaires et d'intégration ont été réalisés :

Tests d'ingestion (load_raw_data.py)

- Tous les CSV chargés → 4 DataFrames.
- Tables non vides et colonnes attendues présentes.
- operation_id présent dans chaque table.
- Vérification de la cohérence entre les tables.

Tests de nettoyage (cleaning/transformation.py)

- Chaque fonction retourne un DataFrame valide.
- Colonnes essentielles conservées (operation_id, etc.).
- Espaces superflus supprimés dans les textes.
- Lignes invalides ou anomalies supprimées, données valides préservées.

Tests de validation (validation/run_validation.py + Pandera)

- Séparation des données curated (valides) / rejected (invalide).
- Respect strict des schémas définis pour chaque table.
- Gestion des cas particuliers : DataFrames vides, fichiers rejetés, valeurs manquantes.

```
===== tests coverage =====
coverage: platform linux, python 3.13.9-final-0
```

Name	Stmts	Miss	Cover	Missing
src/cleaning/transform.py	3	3	0%	1-5
src/cleaning/transformation.py	172	25	85%	74-77, 116, 150-152, 158-160, 216-231, 236-241, 267
src/validation/_init_.py	0	0	100%	
src/validation/run_validation.py	40	3	92%	42, 51, 85
src/validation/schemas_validation.py	8	0	100%	
TOTAL	223	31	86%	

```
===== 32 passed in 4.19s =====
```

11. Conclusion & perspectives

Le projet représente une solution complète et intégrée pour la gestion et l'analyse des opérations maritimes de sauvetage en France. Par son architecture modulaire basée sur le pattern MVC, l'application offre une séparation claire entre la logique métier, la présentation et l'accès aux données, garantissant maintenabilité et scalabilité.

Les tableaux de bord analytiques multidimensionnels fournissent aux utilisateurs une vision 360° des opérations, permettant d'identifier rapidement les tendances, les zones à risque et les profils d'intervention.

L'intégration complète d'un système d'audit garantit la traçabilité totale des modifications, essentielle pour la conformité. Avec sa gestion intuitive des données, ses requêtes SQL optimisées et son interface Streamlit responsive, le projet constitue un outil décisionnel puissant pour les autorités maritimes.

Perspectives et Évolutions Futures

Le projet offre plusieurs axes d'évolution pour les phases suivantes.

D'abord, l'intégration de prédictifs et ML permettrait d'anticiper les zones à risque et les conditions d'intervention critiques.

L'implémentation d'alertes automatiques et notifications temps réel renforcerait la réactivité opérationnelle.

L'expansion vers une API REST faciliterait l'intégration avec d'autres systèmes SECMAR existants et partenaires.

L'ajout de visualisations géospatiales avancées (cartes interactives, heatmaps) améliorerait l'analyse spatiale.

Enfin, une scalabilité horizontale avec cache distribué (Redis) et base de données répliquée assurerait la disponibilité lors de pics d'utilisation. Ces évolutions positionnent l'application comme une plateforme évolutive de référence pour l'intelligence décisionnelle maritime.